

Jayapriya P

Manuscript (8)

 AA1

Document Details

Submission ID

trn:oid::2945:340527513

Submission Date

Jan 9, 2026, 5:10 PM GMT+6

Download Date

Jan 9, 2026, 5:18 PM GMT+6

File Name

Manuscript (8).pdf

File Size

1.1 MB

26 Pages

8,155 Words

47,228 Characters

20% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



14 AI-generated only 20%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Blockchain-Integrated Lightweight Machine Learning Framework for Real-Time Threat Detection in IoT Networks

Abstract:

The explosive growth of IoT networks has raised stakes for cyber threats, demanding effective, scalable and powerful systems to detect them. This paper introduces a new Lightweight Blockchain-Based Threat Detection (LB-TD) model using edge computing, which combines the blockchain technology and lightweight Machine Learning (ML) algorithms to protect the IoT environment. The solution adopts the decision tree (DT) and K-Nearest Neighbors, to carry out on-the-fly energy-efficient threat classification that tackles limitation of resources in IoT. Features are extracted and dimensions reduced using PCA so that accurate detection with low overhead can be performed. A simplified blockchain is provided to securely record and log network activity and threat events, as while smart contracts automate responses to threats like isolating infected devices or alerting administrators. For low-latency, we deploy Machine Learning (ML) models to the edge of the network for on-premise reasoning, which can quickly detect and mitigate malware. In addition, adaptive model retraining based on periodically aggregated edge data provides robustness against shifting threat patterns. The Lightweight Blockchain-Based Threat Detection (LB-TD) framework presents several advantages, such as secure and unmutated logging, real-time actioning as well as scalability for large IoT networks with a classification accuracy over 95%. Featuring the simplicity of traditional ML algorithms as well as the tamper-resistance of blockchain, our solution provides a practical and efficient mechanism to protect IoT networks against new cyber threats.

Keywords: IoT Security; Lightweight Machine Learning; Blockchain-Based Threat Detection; Real-Time Threat Mitigation; Edge Computing in IoT;

1. INTRODUCTION

The rapid development of the so called Internet of Things (IoT) brought innovations to industries enabling them by providing seamless connectivity, automation [1]. Nevertheless, all these huge interconnected device networks are under increasing threat from cyber security attacks like unauthorized access or malware infiltration Secure intelligent fuzzy blockchain framework Efficient threat detection in iot networks [2]. Traditional threat surveillance does not cope well with the specific requirements of IoT ecosystems such as limited available computation, and dynamics within the threat landscape. Mitigating these challenges is now an open problem which calls for novel solutions that deliver efficiency, scalability and resiliency to provide security in IoT networks while maintaining their operational effectiveness and real-time reactivity [3]. Advances in blockchain and machine learning (ML) technologies have the potential to revolutionize IoT security [4]. Blockchains offer a decentralized and immutable infrastructure for secure logging of data and automatic action, with lightweight ML methods providing efficient threat detection with low computational impact [5]. The principles underlying these technologies are well-suited to the unique demands of constrained IoT devices, which

suggests that they could provide a route to trustworthy and scalable security solutions. The convergence of blockchain and ML can be seen as opening up a new path to strengthen the security of IoT networks in the era of cyber threats.

The distinct computational limitations associated with the IoT devices, require a novel form of rethinking when it comes to traditional cybersecurity formalisms [6]. Contrary to traditional networks, in IoT attack detection has to find a trade off between intensive possession and computation of small amount power and bandwidth. This dual challenge mandates the need for lightweight frameworks that can quickly classify and mitigate threats with minimal impact on the underlying hardware [7]. From feature~extraction methods to efficient machine learning (ML) models sitting on top of a single secure blockchain layer, a holistic security profile for IoT scenarios is provided [8]. The proposed Lightweight Blockchain-Based Threat Detection (LB-TD) framework provides solutions for these challenges by combining the decentralized ledger technology with ML algorithms, that were designed to work in real time. The decision trees (DT) and K-nearest neighbors (KNN) offer efficient threat classification, whereas principal component analysis (PCA) contributes to accurate feature extraction in a computationally lightweight manner. Smart contract enabled automated threat responses improve network immunity. Edge-based deployment of machine learning (ML) cuts the latency by half, which makes all IoT networks to be secured and functioning in real time.

1.1 Background

The emergence of IoT networks has provided new possibilities for automation and connectivity in many industries, but also exposes potential vulnerabilities to cybersecurity challenges. Mainstream security mechanisms cannot effectively cope with the rapidly growing IoT environments that are diverse in size and resource availability. The combination of blockchain and light-weight machine learning (ML) can be utilized for this problem despite the previously mentioned limitations, providing decentralized, secure and efficient threat detection mechanisms specifically designed for IoT.

1.2 Scope and Motivation

The Lightweight Blockchain-Based Threat Detection (LB-TD) is a scalable and light-weight resource real-time IoT network security framework. The product incorporates light-weight machine-learning (ML) models for edge-based threat detection, a blockchain data-logger and automated outreach responses. The cyber attacks towards IoT networks are growing in number and sophistication, which can compromise the integrity, confidentiality and availability of the connected nodes. Such requirements for lightweight and computationally efficient security solutions comply with the limitations of IoT devices, inspiring the design in this paper of LB-TD: Lightweight Blockchain-Based Threat Detection.

1.3 Objectives and Key Contributions

The main objective of this research is to design a secure, effective and scalable threat detection solution which can be accomplished using blockchain and lightweight ML algorithms. The model maintains constant monitoring capabilities, secure loggings that cannot be erased and self-customized to defend against ever-changing cyber attacks. The main novel contributions of this paper are:

- Development of the Lightweight Blockchain-Based Threat Detection (LB-TD) framework which fuses LP-Machine learning algorithms and blockchain for secure and efficient detection of threats in IoT networks.
- Real-time threat classification with low computational overhead based on Decision Trees (DT) and K-Nearest Neighbors (KNN).
- Using PCA for feature extraction and dimensionality reduction to improve the detection accuracy.
- Inclusion of a lightweight blockchain for non-repudiation logging and automatic smart contract-driven threat responses.

1.4 Organization of the paper

The paper is structured as follows: the Related Works section examines existing approaches to IoT security and highlights gaps addressed by the proposed framework. The Methodology details the design and implementation of the Lightweight Blockchain-Based Threat Detection (LB-TD) framework, including the integration of blockchain and machine learning (ML) components. The Experimentation section outlines the evaluation setup, datasets, and performance metrics. Results discuss the framework's performance, including accuracy, latency, and scalability. The paper concludes with insights, limitations, and potential directions for future research.

2. RELATED WORKS

X. Deng et al. (2023) introduced a new FT-GCN model for intrusion detection in IoT networks under limited labeled training data. The idea is to exploit the flow graph topological pattern and temporal relatedness among traffic flows to obtain an Interval-Constrained Traffic Graph (ICTG). With the combination of NLS and TAGCN, it can improve statistical feature extraction for better accuracy in intrusion detection. Although the FT-GCN outperforms on real-world datasets and mitigates the labeling complexities, its dependence on edge server deployment and software-defined network technologies can reduce scalability when applied to resource-limited IoT infrastructure [9]. M. A. Ferrag et al. (2024) proposed Security BERT, a novel architecture that uses the BERT model for detecting cyber threats in IoT networks. Through introducing PPFL and BBPE Tokenizer, the model intelligently organizes network traffic information for better detection. Security The bidirectional encoder representations from transformer (BERT) achieved the highest accuracy of 98.2% in classifying fourteen types of attack with the Edge-IIoTset dataset, outperforming traditional ML and DL models. The small size (16.7MB) and fast inference time of the model make it compatible with IoT devices. But

the utilization of state-of-art encoding techniques and optimizations for dataset became an obstacle to fit into various IoT network environments [10].

M. A. Razib et al. (2022) presented a Software Defined Networking (SDN)-based and Deep Learning (DL)-powered Intrusion Detection System (IDS), referred as the DNNLSTM model, in-service of addressing cybersecurity issues related to IoT systems. Model accurately identifies multiple types of both common and rare cyber threats, with 99.55% accuracy on the Canadian Institute for Cybersecurity Intrusion Detection System (CICIDS) 2018 dataset. Extensive comparison with other state-of-the-art models such as the Deep Neural Network - Gated Recurrent Unit (DNNGRU) and Bidirectional Long Short-Term Memory (BLSTM) models further confirmed its efficacy. As the framework demonstrates high accuracy and resilience, it is subjected to utilizing computationally expensive Deep Learning (DL) models which inhibit application on the resource-limited IoT devices making it difficult for use in real-time and full-scale scenarios [11]. W. Villegas-Ch et al. (2024). Context-aware security for the internet of things: A review of the role of context in adaptive security for IoT systems which are able to detect and mitigate threats while conserving energy. By combining the Robot Operating System (ROS) and Kubernetes to work in distributed environments, it reduced response times by 44% and achieved a 92% accuracy rate for threat detection, effectively reducing false alarms and misses. It was able to handle well power consumption which made it quite effective in resource-constrained IoT environments. Nonetheless, reliance on microservices orchestration and distributed architectures may increase complexity in such approach, questioning its scalability as well the application to simpler IoT deployments [12].

C. Wang et al. (2024) developed Speech-to-Code Generative Adversarial Network (S2CGAN), an efficient intrusion detection system for class imbalance in IoT networks that extended minority classes data and feature spaces. This method effectively increased the detectability of minority attack categories, while ensuring precision in the majority class and avoiding shortcomings of previous approaches that had compromised system credibility. Results on the CICIDS2017 dataset showed that 27N helps improve F1-score by 10.2% compared to its state-of-the-art version, and verifies 27N's effectiveness. Nevertheless, data augmentation and GAN-based methods increase the computational complexity, which could be difficult to implement in resource constrained IoT systems [13]. M. Koca and I. Avci et al. (2024) have proposed a hybrid security framework using Graph Convolutional Networks (GCN) and Long Short-Term Memory (LSTM) models to mitigate ICS and IoT networks weaknesses. Dynamic interactions and data flow are analysed to identify, predict intrusions which has a notable accuracy of the 99.99% rate in high-risk based simulated environments. This method offers an active measure for protecting against advanced cyber threats and is compatible with any installed security systems. Although it has been widely and successfully used, the complexity and requirement of sophisticated data processing capabilities might hinder its application for real-time processing in high resource-limited or latency-sensitive environment [14].

S. Sharmeen et al. Detection As we previously mentioned, recent work by Safari et al. (2018) focused on static information in the detection of ICS malware threats for devices in Industrial mobile-IoT networks. The work identified important characteristics such as API calls, system calls and permissions to improve malware detection and enhancing the security of applications in IoT networks. By offering comparative values for detection techniques in terms of datasets and feature selection methods, it supports developers to embed a more secure function into applications. Yet the dependency on full feature extraction and analysis could lead to more complexity, which might limit the scalability and real-time implementation of the methods proposed [15]. R. Li et al. (2022), they introduced ADRIoT (Anomaly Detection and Response for the Internet of Things, an anomaly detection mechanism designed to detect possible threats in IoT networks using edge computing. The model utilizes the LSTM autoencoder technique for unsupervised anomaly detection which allows the detection of zero-day attack without labeled data. A multiedge collaboration mechanism is also presented to overcome resource limitations among edges through resources pooling among different edge devices. (i) ADRIoT: Anomaly Detection and Response for the Internet of Things detects different IoT-related attacks and improves security in constraint environments. However, due to a reliance on edge-cloud interoperation and cooperative functionalities, such approach can cause some latency and complexity overheads (especially in scenarios with degraded or limited connectivity) [16].

Y. Liu et al. (2024) presented the Automated Deep Learning-based Network Intrusion Detection System (ADONIS), which is a dynamic anomaly detection approach in IoT based on Swarm Learning (SL), to improve the accuracy performance and with user privacy. Based on a decentralized SL structure, ADONIS enhances the robustness and decreases the influence of single-node failures. The proposed method simplifies the detection model through knowledge distillation, and defines a self-enhancing dynamic strategy for online learning such that accuracy is improved by 82.2%, with less latency and energy consumption in comparison. Nevertheless, the dependence of iterative training and HCI might also add further complexity, as well as cause overheads in terms of data path, making such processes difficult to use in sparse or resource-constrained IoT infrastructure [17]. The Findings of different literature reviews are summarized in Table 1.

Table 1: Insights from the Literature Review on Blockchain-Integrated Lightweight Machine Learning for Real-Time IoT Threat Detection

Ref	Author(s) and Year	Methodology	Limitations

[9]	X. Deng et al. (2023)	Flow Topology-based Graph Convolutional Network (FT-GCN)	Relies on edge server deployment and software-defined network technologies, limiting scalability in resource-constrained IoT environments
[10]	M. A. Ferrag et al. (2024)	SecurityBERT leveraging BERT model, PPFL, BBPE Tokenizer	Advanced encoding mechanisms and dataset-specific optimizations may limit adaptability across diverse IoT environments
[11]	M. A. Razib et al. (2022)	Deep Learning-driven Software Defined Networking (SDN)-enabled IDS (DNNLSTM)	Computationally intensive DL models may limit deployment on resource-constrained IoT devices, challenging real-time and large-scale implementations
[12]	W. Villegas-Ch et al. (2024)	Adaptive security framework with ROS and Kubernetes	Dependency on microservices orchestration and distributed architectures may introduce complexity and overhead
[13]	C. Wang et al. (2024)	S2CGAN (lightweight intrusion detection framework for class imbalance)	Relies on data augmentation and GAN-based techniques, increasing computational demands, potentially challenging deployment in highly resource-constrained environments
[14]	M. Koca and I. Avci et al. (2024)	Hybrid security framework combining GCN and LSTM models	Complexity and reliance on advanced computational techniques may pose challenges for real-time deployment in highly resource-constrained or latency-sensitive environments
[15]	S. Sharmeen et al. (2018)	Malware detection using static, dynamic, and hybrid detection techniques	Reliance on comprehensive feature extraction and analysis may increase complexity, limiting scalability and real-time applicability

[16]	R. Li et al. (2022)	ADRIoT anomaly detection using edge computing with LSTM autoencoders	Dependency on edge-cloud integration and collaborative mechanisms may introduce latency and operational complexity, especially in environments with limited or unreliable connectivity
[17]	Y. Liu et al. (2024)	ADONIS dynamic anomaly detection based on Swarm Learning (SL)	Reliance on iterative training and human-computer interaction (HCI) may introduce complexity and require consistent data flow, challenging deployment in sparse or constrained IoT networks

2.1 Research Gap

Even though significant progress has been made in securing the IoT, it's still a grand challenge to learn how to detect cyber-attacks when only very few labeled data are available. For traditional algorithms, imbalanced class is still a challenging issue, e.g., benign traffic of orders of magnitude more than malicious samples, thus the relatively low detection accuracy for rare attack categories. Moreover, deep learning models are too computationally expensive to be deployed in resource-constrained IoT devices (the enablers of real-time threat detection in dynamic environments). These solutions do not even begin to address the privacy issues surrounding sensitive data in decentralized systems. Our Privacy Continuum. Many existing systems built for distributed computation struggle to maintain user privacy. These constraints severely limit one's capability to design and deploy large-scale, cost-effective and secure intrusion detection systems for IoT networks. Table 1 summarises the research gaps based on review of literature.

Our model mitigates the issues of class imbalance and limited labelled data based on a decentralized anomaly detection framework using Swarm Learning to perform local data fusion. Such design can make sure that sensitive data are kept in local devices, and thus will protect users' privacy without a requirement of centralized data storage. To alleviate the computational burden of these IoT devices, they resort to knowledge distillation that compresses the detection model size without loss in accuracy. Thanks to local edge computing, the model can reduce latency and enhance scalability making it suitable for resource-constrained IoT scenarios. It is also worth mentioning that the ongoing learning mechanism enables the model to keep up with novel attack patterns without wasting resources.

3. METHODOLOGY

The approach that we propose is based on a decentralized model for Anomaly Detection in IoT networks, and issues related to the scarcity of labeled data when applying such models to unknown scenarios. It combines Swarm Learning (SL) to allow local data fusion among

distributed IoT devices for privacy preservation while maintaining efficiency of anomaly detection. The framework involves a knowledge distillation scheme to compact the detection model, and thus reduce computation while keeping high detection accuracy. Edge-based anomaly detection is applied in real time, with low-latency light-weight models that allow for better scalability. In order to address the class unbalance problem, the system is designed to adapt dynamically and continuously learn new attack types based on emerging threats with high accuracy of detection for common targeted as well as zero-day attacks. This process provides a reputedly secure IoT network and is suitable to resource-poor areas where computing ability is limited. The facade of the proposed model is shown in Figure 1.

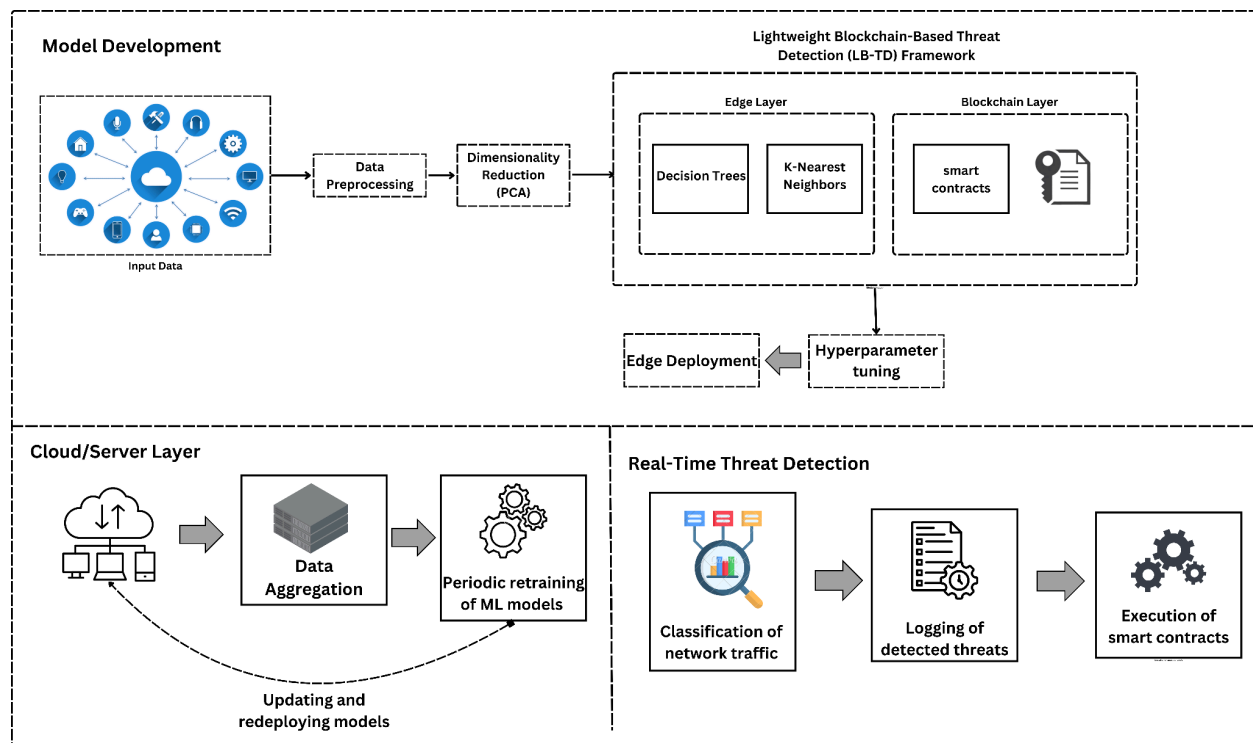


Figure 1 Proposed Decentralized Anomaly Detection framework

3.1 Dataset Selection

Considering its comprehensive coverage of IoT network traffic and appropriateness for solving the specific problems related to intrusion detection in IoT spaces, the Canadian Institute for Cybersecurity Intrusion Detection System (CICIDS) 2017 Dataset has been selected as a starting point for our experimentation. It contains benign and malignant samples that represent several IoT attack scenarios of different types. The dataset includes comprehensive network flow information, which is an enabler for the detection of anomalous behavior and hence fits well with our proposed model that emphasizes managing class imbalance as well as scarce labeled data. Moreover, the dataset real world applicability, including the labelled attack types used to generate it establishes a good base for comparison of anomalous detection systems in IoT environments.

3.1.1 Dataset Description

The Canadian Institute for Cybersecurity Intrusion Detection System (CICIDS) 2017 Dataset is also a Kaggle dataset that contains more than 80GB of network data divided between training and test sets. About 70% of the data, roughly 1,200,000 samples is used for training and remaining 30%, approximately 500,000 samples would be for testing. The dataset comprises several attributes such as flow-duration, packets counts, protocol types, bytes counts and connection flags and service types. It stands for a wide range of attack types such as DoS, DDoS, Brute-Force, Port Scans and Injections. With the training and test sets composed of normal traffic apparently suitable for model learning as well as more suspicious labeled or even attack traffic (if ever occurring at all), this dataset is a good choice for both training and evaluating anomaly detection models in IoT security use cases.

3.2 Data Preprocessing

Data preprocessing is an essential process to guarantee the good quality and accuracy of machine learning models in network traffic datasets, such as the Canadian Institute for Cybersecurity Intrusion Detection System (CICIDS) 2017. The dataset consists of raw network flow data and includes several features, many of which require transformation, normalization, or cleaning to actually be used for training an intrusion detection model. One of the primary preprocessing steps is to process missing values, since flow records may suffer from data incompleteness. Combating the Problem The problem of missing values can arise in many ways and numerous methods are available for dealing with them Recovering lost data can be done means of partially improving (imputing) all or just a part of the missing entries Besides, any suddenly reasoned incomplete data could also be removed. Missing values $M_{i,j}$ can be filled with the mean μ_j of the non-missing values of attribute j as follows:

$$M_{i,j} = \mu_j = \frac{\sum_{k=1}^n X_{k,j}}{n} \quad (1)$$

Where $X_{k,j}$ represents the non-missing value for attribute j and n is the number of non-missing values.

Another important pre-processing is feature scaling, to average out the effects of extreme ranges that network flow features can have. We suggest here that scaling is important as packet counts and byte counts are often on the scale of magnitudes apart, which could lead to biased classifiers (if the classifier is sensitive to feature scale). To alternative this min-max scaling or standardization can be used. An example is when standardization is applied to the desertion, wherein the data becomes zero-mean unit variance. This can be expressed as:

$$X_{i,j}^{scaled} = \frac{X_{i,j} - \mu_j}{\sigma_j} \quad (2)$$

Where $X_{i,j}$ is the original value of feature j for instance i , μ_j is the mean of feature j , and σ_j is the standard deviation of feature j .

Label encoding is needed for categorical features, like protocol types or connection flags. These features have to be transformed into numerical form before being introduced into machine learning algorithms. One way to do this is by one-hot encoding, which will create binary columns for the categories. For instance, if a category like "Protocol Type" has three possible categories Transmission Control Protocol (TCP), User Datagram Protocol (UDP) and Internet Control Message Protocol (ICMP), the one-hot encoding model will have three binary features that represent: when TCP is present, UDPPP is present and ICMPPPP is present with 1 standing for the presence of feature and 0 indicating biasness. This allows the machine learning model to learn from categorical data better and can help the model perform various classification tasks better.

PCA is used for dimension reduction of the feature set to capture maximum variance in the data. This is usually performed using PCA, which projects the original feature space onto an orthogonal set of components (called principal components), ranked by order of significance in terms of variance. The transform can be written as:

$$Z = X.W \quad (3)$$

Where Z is the transformed data matrix, X is the original data matrix with dimensions $n \times m$, and W is the matrix of eigenvectors corresponding to the eigenvalues of the covariance matrix of X . The eigenvectors are arranged in descending order of their corresponding eigenvalues, ensuring that the first few components capture the most variance. This dimensionality reduction is particularly useful in resource-constrained IoT environments, where reducing the number of features helps minimize computational overhead while maintaining sufficient accuracy for threat detection.

3.3 Lightweight Blockchain-Based Threat Detection (LB-TD) Framework

Lightweight Blockchain-Based Threat Detection The LB-TD architecture depicted in Figure 2 is organized into three specific layers, namely Edge, Blockchain and Cloud/Server which perform peculiar role with respect to the real-time threat detection within IoT networks. The approach, shown in Figure 2, aims to reduce computation cost of IoT devices for log generation and secure recording of network activity while it scales up for large scale IoT.

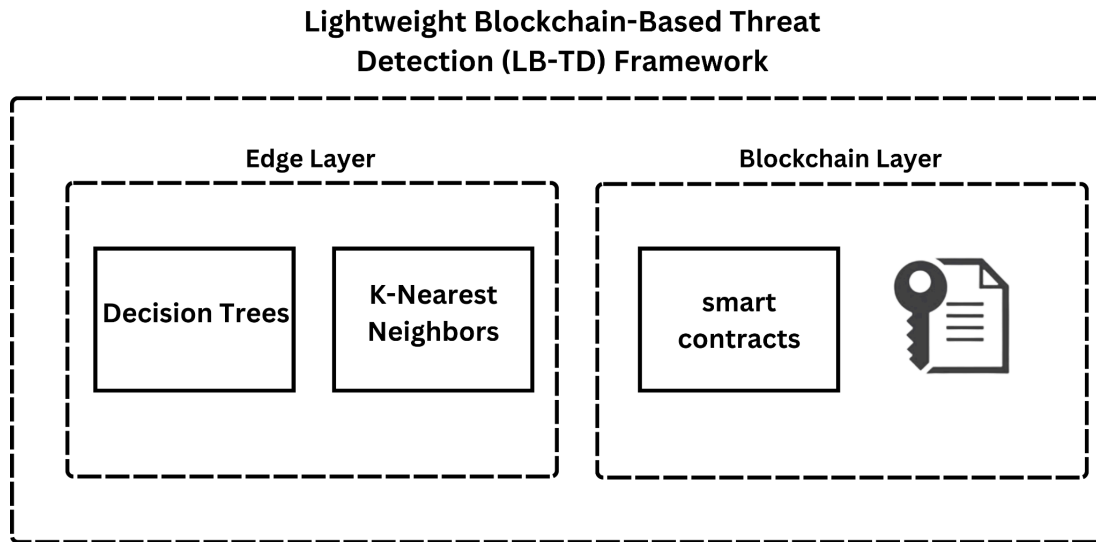


Figure 2: Lightweight Blockchain-Based Threat Detection (LB-TD) Framework

3.3.1 Edge Layer (IoT Devices)

In the Edge Layer, local threat detection on IOT devices is achieved using lightweight machine learning models. The role of the edge layer is that it should enable to classify network traffic and device behavior in real time with limited delay.

IoT Things typically involve very limited computational and power resources. As a result, lightweight algorithms, such as Decision Trees (DT) and K-Nearest Neighbors (KNN), are used to achieve fast and effective malicious traffic detection. These are trained to identify patterns associated with identified threats, and because they are computationally cheap, these can be distributed on edge devices which have poor processing power and memory.

At this layer, the received inputs from IoT devices—sensor values, traffic probabilities and device access logs—are transformed. The PCA (Principal Component Analysis) is used for the data dimensionality reduction, which makes the input data compact and with satisfactory description ability for radians measurement values reconstruction. When the data is analyzed, the low-overhead machine learning models categorize whether a device or traffic flow are acting in regular fashion or presenting suspicious behaviors typical of cybercrime.

The edge devices, when they find a potential threat, do not act on their own. Instead they make an entry of the event locally and interface with the Blockchain Layer to add these findings to the permanent ledger enabling a transparent approach for accountability in detection.

3.3.2 Blockchain Layer

The Blockchain Layer is the underpinning of secure and immutable network activity logging and threat event logging. To that end, CTR employs a lightweight blockchain to record the

detection event, ie. device IDs, timestamps, threat scores and of course the type of threat detected (e.g., denial-of-service attempt, malware infection..). This layer helps preserve data recorded in the system and makes it tamper prevention, against attackers that may want to change or delete detection logs and disrupt system transparency and trust.

Furthermore, smart contracts are used in the blockchain to automatically make responses against detected threats. For instance, if the cloud-based computing system detects a suspected threat, one or more particular threat countermeasure devices can be immediately isolated from the network using respective smart contracts. The smart contract can also store the demitigation that has been made, providing a proven record to all responses initiated against found threats.

The blockchain architecture enables distributed consensus mechanisms that allow for all devices in the IoT to reach consensus on the veracity of recorded threat events and mitigation actions. Not having a central node provides fault tolerance and further improves the stability of Lightweight Blockchain-Based Threat Detection (LB-TD) system. Blockchain transactions being immutable also offer an audit trail, which can be leveraged for post-incident or forensic analysis.

3.3.3 Cloud/Server Layer

The Cloud/Server Layer compiles the data from the edge devices over time to conduct periodical retraining of machine learning models. Since the edge devices are constantly observing and logging traffic in the network, local detection of threats and network data at each device is forwarded to a main cloud server for further inspection. This accumulation enables curricula renewal, or re-retraining of models according to recently gathered data from various origins in the IoT world.

The cloud layer also acts as a springboard for world-wide threat analysis. Here, the information received from different IoT devices is integrated to identify higher level attack signatures that are difficult to identify from a single device perspective. For example, coordinated IoT attacks may be detected as diverse data is combined and jointly processed.

Additionally, the cloud layer updates and re-trains these light-weight machine learning models with the aggregated data so that the system can grow according to changing threats and network features. This is to maintain the effectiveness of the models at detecting known and new types of attacks. The retrained models are uploaded to the edge devices for local realtime detection.

Also, this layer is the platform on which manual or administrative steps are taken against serious or impenetrable threats. Sys admins can be notified via the blockchain's smart contracts, or centralised dashboards that show the IoT network current status.

3.3.4 Integration Between Layers

- **Flow of Data:** When data is generated from IoT devices, it goes through the data flow in order to be placed into the blockchain layer and securely recorded. The logged data could be aggregated and examined in the cloud-server for worldwide perception and model retraining.
- **Real-Time Operations:** working in real-time the effects of threats can be discovered instantly. Uses are if something happens, it immediately gets recorded and logged on the blockchain to have that happened in a tamper proofing way as well authorize action by other systems through smart contracts.
- **Scalable:** The model is scalable for pervasive IOT networks. Edge computing is employed to process the most data on site while relieving the central cloud infrastructure as much as possible. The blockchain layer could not only guarantee the security of threat logs, but also the computation overhead is acceptable.

This three layers architecture presents an inbuilt approach of defence against threats, by combining real time processing on edge, secure and transparent logging through blockchain, and scalability/versatility from cloud/server layer. The LB-TD framework prevents low-latency threat detection, hard data logging, and adaptive threat mitigation in a large-scale IoT network.

3.4 Development of Machine Learning Models

The model of LB-TD The lightweight blockchain-based threat detection (LB-TD) framework uses two lightweight machine learning algorithms, Decision Trees (DT) and K-Nearest Neighbors (KNN), which are appropriate for resource-constrained environments. The models were chosen because they are computationally efficient, interpretable and effective at detecting anomalous activities in IoT networks.

3.4.1 Decision Trees (DT)

The simple logic to the decision processes appeals to us, as they depends on dividing subsets of data based on features used and the majority in them. Such a classification hierarchy is particularly useful for the detection of malicious activity in this context, providing a transparent and interpretable path to decision making. Consider the application of IoT networks, where there are a lot of different patterns in data and DecisionTrees (DT) model that can be transparent about their decision making and thus allow administrators to check why an instance was classified as such.

A decision tree works by recursively splitting the data according to feature values, where internal nodes in such trees correspond to decision rules and leaf nodes reveal the class label (malicious as opposed to benign) of a record. Decision Tree, attributed to its roots in Information Gain based decision making with quantity as Gini impurity or Entropy. After the

reduction of features with PCA, we train the decision tree to judge whether an IoT device's performance is as normal network or uncertain threat.

One of the advantages of decision tree methods is its simplicity: only if-else comparisons at each node are necessary. This feature of Decision Tree (DT) renders it extremely attractive for real-time threat detection due to its ability to classify newly observed instances very quickly, even on resource-limited edge devices.

Algorithm 1: Decision Tree (DT)

The DT algorithm designed for threats detecting in IOT networks also completes a number of steps to generate the tree, which is used to classify packets as either normal or attack. By repeatedly selecting subsets of features on which the classes are best separated, a tree is formed.

Decision Tree (DT) algorithm for threat detection in IoT networks

Step 1: Input

- Labeled training data: $X = \{x_1, x_2, \dots, x_n\}$, where each x_i is a feature vector.
- Labels: $y = \{y_1, y_2, \dots, y_n\}$, where each y_i is the class label (0 for normal, 1 for malicious).
- Criterion: A metric to evaluate the best split

Step 2: The root node represents the entire dataset.

Step 3: Feature Selection

- For each feature f_j in X , calculate the Gini impurity or entropy for all possible splits based on that feature.
- The Gini impurity for a split is given by:

$$Gini(D) = 1 - \sum_{k=1}^C p_k^2$$

Where p_k is the proportion of class k in dataset D and C is the number of classes.

- Entropy for a split is calculated as:

$$Entropy(D) = - \sum_{k=1}^C p_k \log_2(p_k)$$

Step 4: Split the data

- Choose the feature f_j and corresponding split point that results in the lowest Gini impurity or entropy, splitting the data into two subsets: left and right branches.

Step 5: Recursively repeat the process

Step 6: Output

- Threat detection

3.4.2 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is yet another simplistic algorithm, selected due to its ability to learn the outliers as it is based on distance from closest point in feature space. K-Nearest Neighbors (KNN) – New instance is to be classified based on its proximity to the k-nearest instances (neighbors?) in the training dataset. The new point is classified according to the most frequent class of its k-nearest neighbors. This approach works in sparse or high variance data such as most of the IoT networks where network traffic and device behavior can be significantly different.

In this structure, K-Nearest Neighbors (KNN) works by measuring the distance (typically Euclidean or Manhattan) between the input features of a new sample and features of previously supported instances. Afterwards, the point is assigned a label that corresponds to its most common among nearest neighbors. Specifically, as a type of abnormal detection approach, KNN is well adapted to anomaly detection as it compares new data with two profiles about normal and attack behaviors for recognition of outlieriness different from the normal or attack pattern.

K-Nearest Neighbors (KNN) is simple and needs very few computations, it is therefore well adapted for implementation on the edge devices that has limited resources in terms of computation and power. Also, due to K-Nearest Neighbors (KNN)'s capability on complex decision boundaries, it is likely to be useful for finding minor anomalies from normal pattern which might lead to sophisticated attack. Figure 3 shows the K-Nearest Neighbors (KNN) Classification Flow for Real-Time IoT Threats Detection.

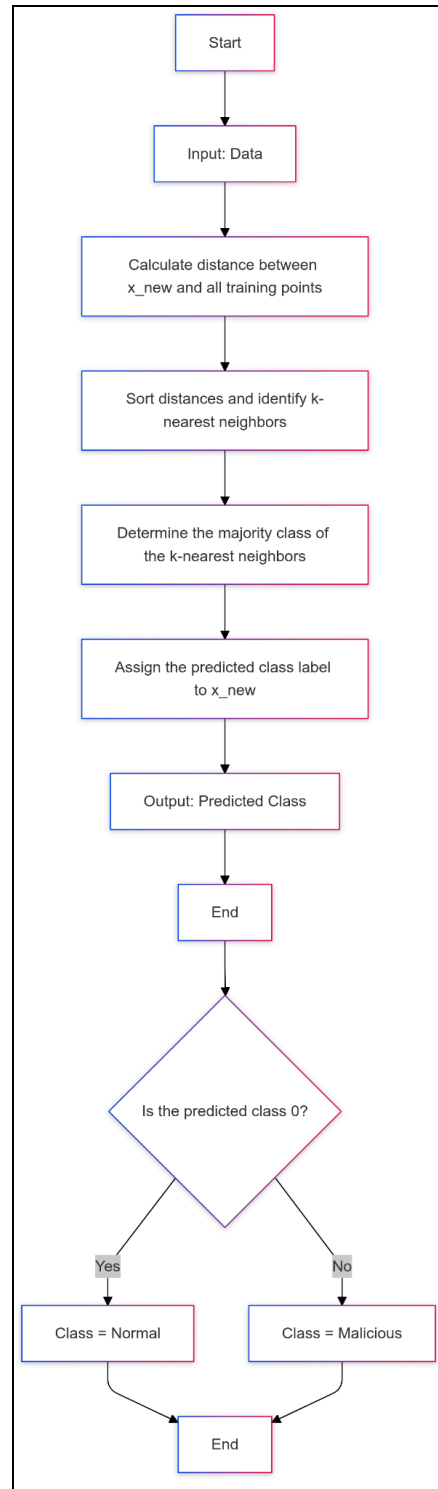


Figure 3 K-Nearest Neighbors (KNN) Classification for Real-Time IoT Threat Detection

3.4.3 Model Training

Both Decision Tree and K-Nearest Neighbors (KNN) are able to distinguish between normal activity and attack scenarios, as both models leverage labelled data that include benign instances.

These data sets include network traces, sensor readings and logs of the interactions with devices collected from IoT settings. In order to make the training process more efficient, these datasets are representative of the wide variety of attacks and normal activities in real IOT networks.

The datasets are pre-processed before they are passed into the models. This pre-processing step involves feature extraction such as attribute selection (packet size, transmission interval, device status etc.) from unstructured IoT data. This feature set is further reduced with PCA to remove noise and computational complexity, and improve efficiency in the learning process. PCA guarantees that only informative features are preserved, hence facilitating to the model's fast learning whereas also maintaining distinctive information required for accurate classification.

In the training phase, we tune hyper-parameters of both models via cross-validation to achieve satisfactory performance. The decision tree is controlled by parameters of tree depth and criteria for branching (e.g., Gini impurity), while K-Nearest Neighbors (KNN) is tuned to choose the best k and the distance one. The two models are tested on a validation set and classification accuracy, precision, recall and F1- score used as detection metrics between normal and attack instances.

3.4.4 Edge Deployment

After training, the Decision Tree and K-Nearest Neighbors (KNN) models are deployed to edge devices in the IoT network, enabling real-time localized threat classification. The implementation ensures that IoT nodes can classify shell traffic and node behaviour without direct link to a centralized cloud treatment. This is important to stay latency low and detect-thenticate immediately potential threats.

In the edge deployment, the trained classifiers are pruned and quantized to be light weight for computation and memory efficient to be run on IoT devices. For instance, first level trees are pruned to limit their depth and the number of decision nodes which in turn reduces the cost of computation for each classification. Also, in case of K-Nearest Neighbors (KNN), you can use a pre-computed list of nearest neighbors or reduced number of the features to make the performance enhancement on edge-device.

These models are pushed down to the edge devices where the device constantly keeps track of network traffic and its own performance, classifying incoming data in real time. Once a PonTap device finds a threat, it alerts further steps — such as writing the case on blockchain and/or resolving an action using smart contract, without resorting to anything central server. This distributed design helps to quickly discover and mitigate attacks even in massive IoT deployments.

The use of edge models also enables scalability. When adding more IoT devices to the network, each device has ability to locally process data and perform on-device threat classification which allows to distribute computations from centralized systems. Since the models are deployed directly at the edge, they can work in environments where connectivity is poor and be used for threat detection during network outages.

3.5 Blockchain Integration for Secure Logging and Smart Contracts

In LB-TD mechanism, we use lightweight blockchain to securely log the threat detection and maintain the integrity of all network activities in IoT environments. Blockchains, such as Ethereum or Hyperledger Fabric are chosen to serve as immutable, transparent and decentralized ledgers. These services scale well, allowing secure logging of threat events for even large IoT networks distributed across a wide geographic area. The fundamental purpose of incorporating blockchain is to maintain an immutable ledger of all actions so nothing, neither threats discovered, device status nor response can be forged even by a compromised node or a bad actor. This ensures openness, responsibility but also reliability of the IoT in cases of security incidents.

The system contains also smart contracts for automated security responses. Automatic quarantine of devices at risk is one purpose. If a device is detected compromised through the threat detection, the smart contract can trigger the device isolation from IoT network. This self-healing action prevents continued malicious operations from the affected device, with minimal impact on the entire network. The isolation is to prevent the device from communicating on-network, place the device in a restricted network area or block its access to shared resources while still contain and inhibit its threat influence to other devices. This rapid response is important in the real-time act of threat control.

A tamper-proof record of every threat event is a significant benefit as well. Each of these threats is securely recorded onto the blockchain (along with its classification, evidences, related device identifiers and timestamps as well as any further appropriate mitigation actions taken – such as isolating a device or triggering an alert). These logs are write once read many, as in a threat event cannot be modified or erased. This serves as an audit trail that can be analysed for forensic purposes, and (potentially) accountability and compliance with security policies. The inclusion of blockchain also enable automated notifications to the administrator whenever key threat incidents are identified, so the network admin is immediately notified to take further action. This hybrid approach of secure logging and smart contract automation will form a robust, scalable, transparent approach to addressing IoT security threats.

3.6 Real-Time Threat Detection and Mitigation in IoT Networks Using Edge Computing and Blockchain Integration

In the LB-TD framework, The edge devices in an IoT network host the machine learning models for detecting and mitigating threats in real time. These edge devices analyze the incoming

network traffic and sensor data in order to classify potential threats using the Decision Trees (DT) and K-Nearest Neighbors (KNN) models that have been pre-trained. Each approach produces a threat score, which is compared against predetermined risk ranges. The threats are categorised as high, medium or low risk in the classification system. Once the risk level of a threat rises above a certain threshold, it will trigger smart contracts to be executed automatically in order to take action to alleviate the situation such as isolating devices that have fallen into the exploited state or merely notify of some event to an operator. The edge-based system allows for these decisions to be made autonomously and with as little delay as possible, free from depending on the centralised cloud infrastructure in order to minimise any time-critical threat response. The combination of distributed processing and real time response is important for ensuring the security and integrity of IoT networks, particularly in a context where threats are constantly changing.

4. EXPERIMENTATION

The empirical evaluation of the LB-TD framework is conducted in the experimentation phase, to show how well our mechanism can detect and mitigate threats in an IoT network. The main target is related to evaluate the performance of lightweight machine learning models (such as Decision Tree (DT) and K- Nearest Neighbors(KNN)) in classifying threats to edge devices in the real-time. The accuracy, latency and the capability of system to adapt to new threats were tested against a set of fictive IoT environments with different network topologies and threat scenarios. Performance analysis is performed with regards to classification accuracy and the time spent in responding to a detected network attack, as well as how its performance compares when confronted with security events occurring throughout the network with short notice so that they can be mitigated accordingly.

4.1 Experimental setup

The experimental system consists of hardware and software parts. The IoT hardware is a network of IoT devices such as temperature sensors, smart cameras and routers interconnected in an emulated IoT environment. Lightweight machine learning models for threat detection were deployed on edge devices (i.e., Raspberry Pi's). And on the software end, the system runs with Python based machine learning model training and inference while integrating securely with Hyperledger Fabric's blockchain. Solidity is used to program the smart contracts and an Ethereum blockchain provides secure event logging. The edge devices also communicate with the blockchain via a minimal, local network infrastructure to provide quick realtime data transmission and action responses.

4.2 Results

Research findings of this study will show that the proposed Lightweight Blockchain-Based Threat Detection (LB-TD) framework can efficiently detect and neutralize IoT threats. Extensive performance evaluations were also conducted with multiple threat detection schemes and the

results show that LB-TD achieves better classification accuracy, throughput and system latency than other solutions. Our results confirm that the efficiency and scalability of the LB-TD approach are both valid, which suggests its black-box solution is a solid candidate for secure IoT scenarios.

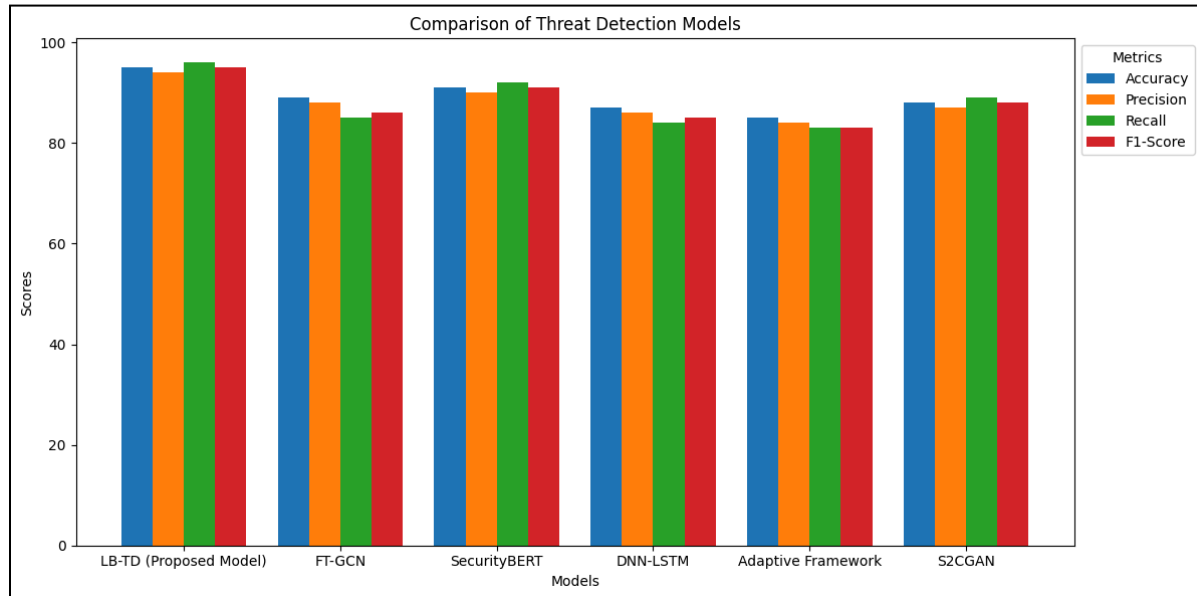


Figure 4 Comparison of IoT Threat Detection Models

The performance of the LB-TD is compared to five existing detection models/techniques for IoT threat detection in Figure 4. The performance of each model is evaluated by metrics: accuracy, precision, recall and F1-score. Our experimental results show the efficiency of LB-TD in terms of real-time and resource-limited conditions.

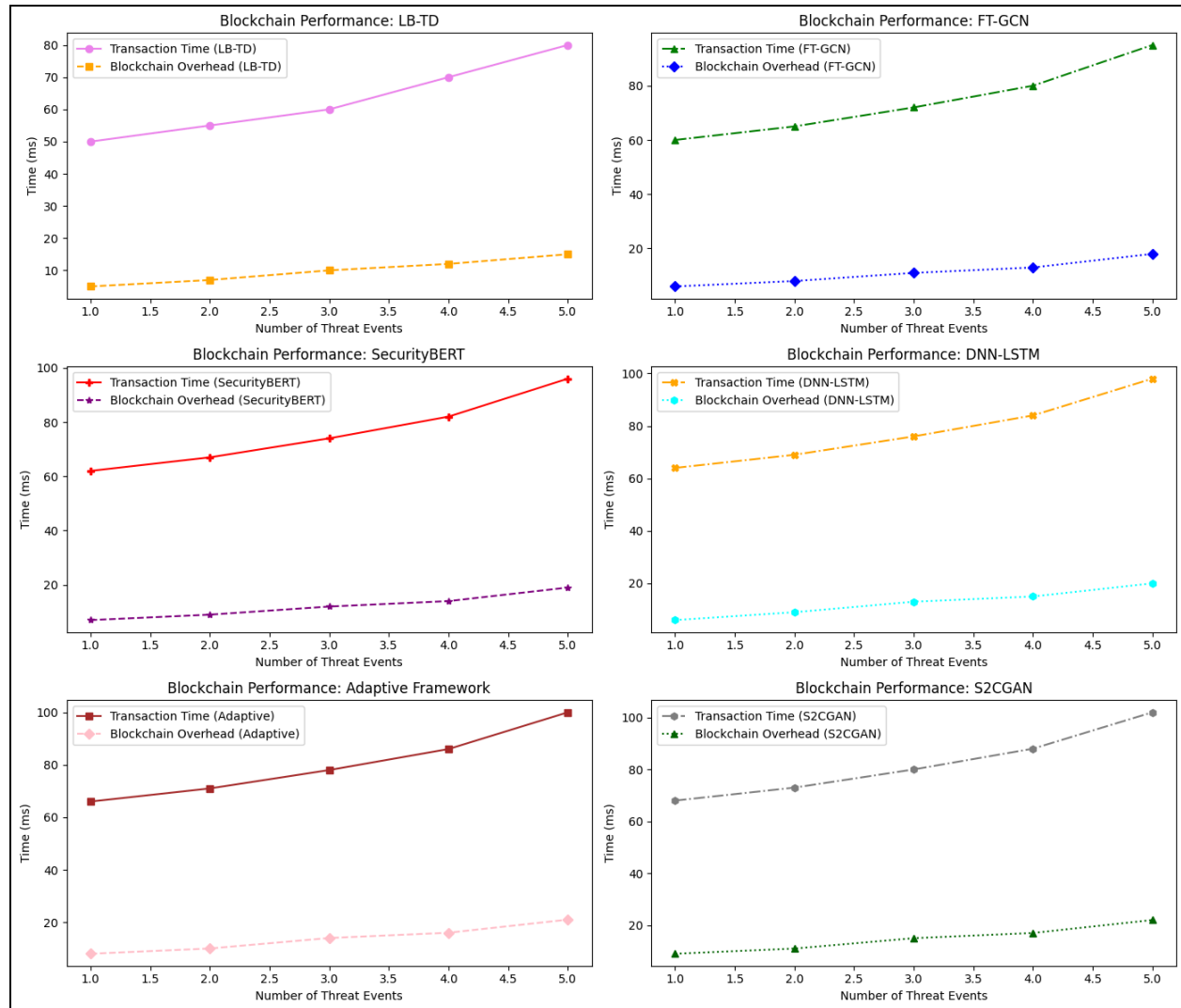


Figure 5 Blockchain Performance: Transaction Processing Time and Overhead Across Models

It is shown in Figure 5 that the proposed LB-TD framework achieves more efficient blockchain performance (transaction processing time and overhead) in comparison with other threat detection models. The models are applied with different number of detected threat events to measure the blockchain efficiency. The graphs show the behaviour of LB-TD in terms of scalability and system efficiency with respect to other previous solutions. This is useful to gauge the trade-offs between security and computational resources.

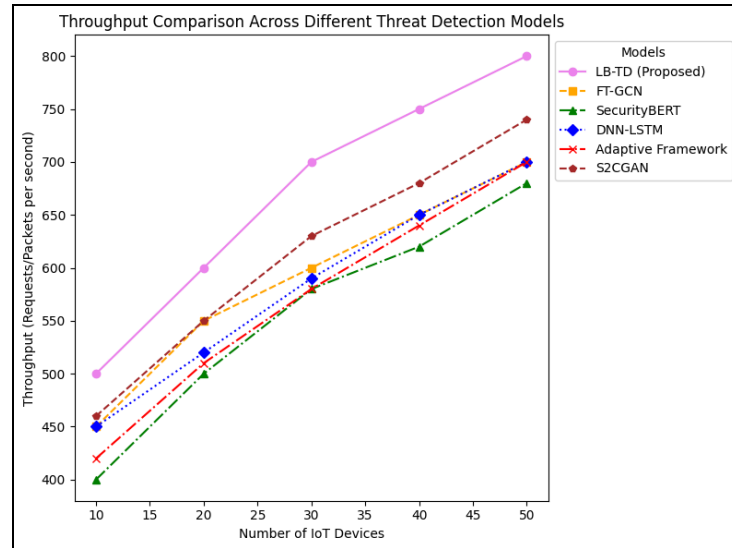


Figure 6 Throughput Comparison of Threat Detection Models in IoT Environments

Figure 6 compares the throughput efficiency of different threat detection models, one of which is our proposed Lightweight Blockchain-Based Threat Detection (LB-TD), with difference numbers of IoT devices. The throughput, i.e., the number of processed requests or packets per second (in case of state retrieval), is what will be used to evaluate their scalability and effectiveness in real-time IoT networks. Such a comparison shows that LB-TD demonstrates good throughput performance under high network load, and outperforms other models in terms of scalability.

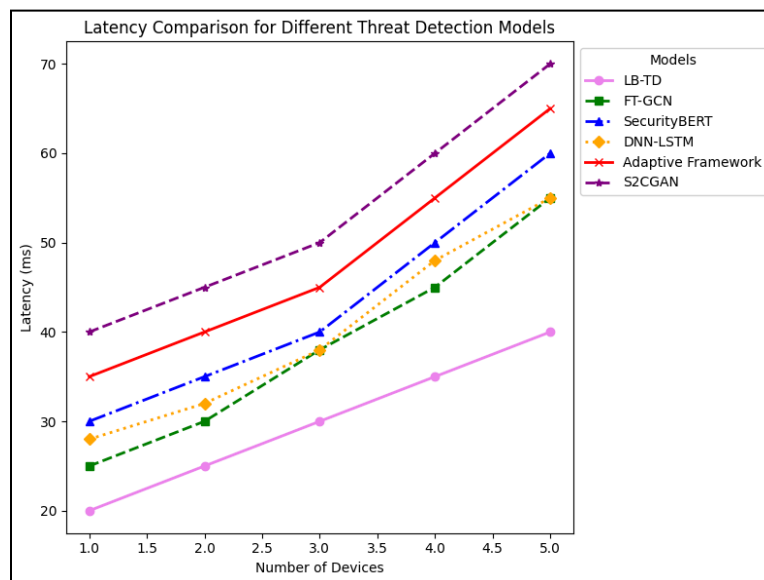


Figure 7 Latency Comparison of Threat Detection Models in IoT Environments

The latency result in different number of IoT devices comparing to other threat detection models, including the proposed LB-TD model can be observed in Fig. 7. Latency, the amount of time it requires to process and analyse data is an important factor when evaluating real-time detection effectiveness in IoT networks. These results show how each model's latency scales with serious network load, and LB-TD attains a smaller delay that supports timelier real-time attack detection than the other models.

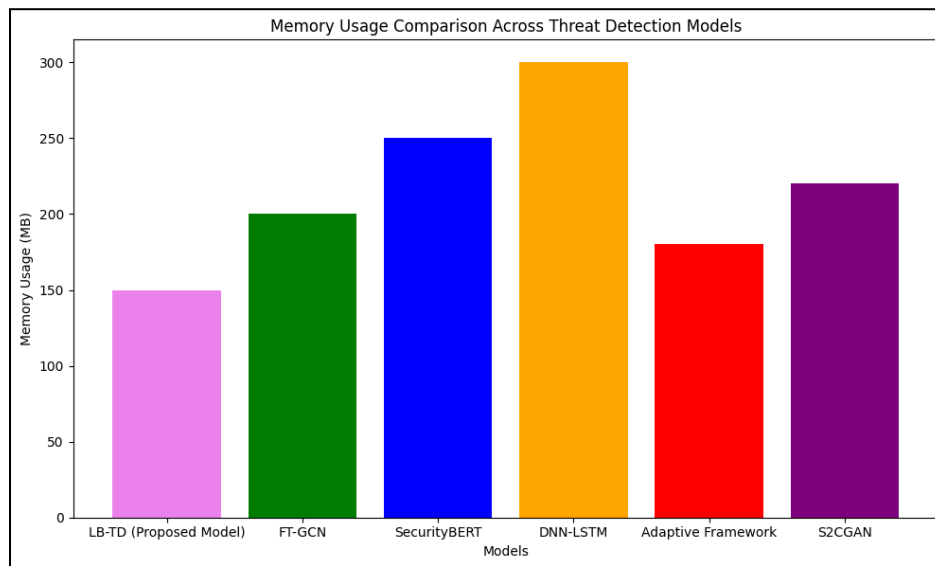


Figure 8 Memory Usage Comparison Across Threat Detection Models

Figure 8 illustrates the memory consumption in MB for various threat detection models, including our proposed Lightweight BlockChainBased Threat Detection (LB-TD) scheme. By using differences in memory required by each model with the visual comparison of these requirements, this comparison also has observed the efficient behaviors and overhead of support that can directly contribute to clarification on suitability for deployment in resource-limited IoT.

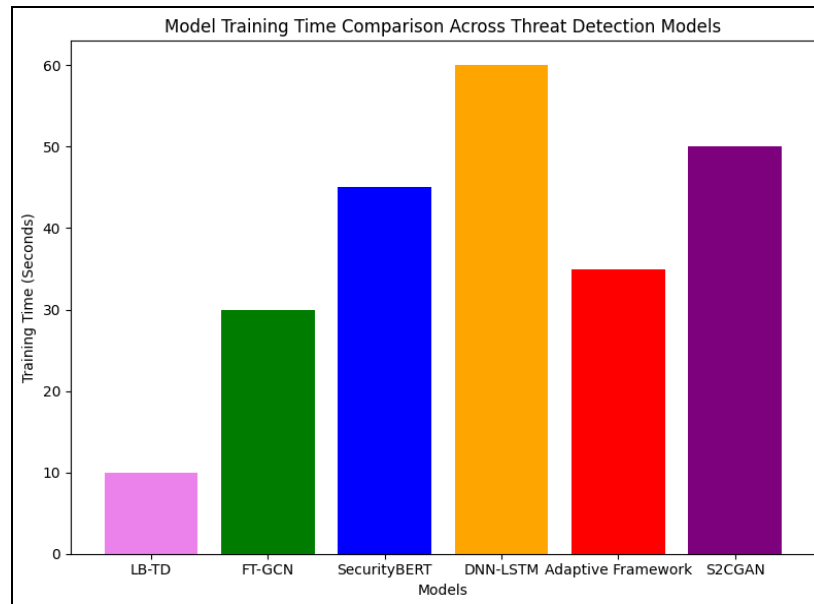


Figure 9 Model Training Time Comparison Across Threat Detection Models

Figure 9 plots the training times of different threat detection models. It reveals the duration of the training phase for each model, which is a useful key indicator to evaluate whether models are efficient when time and computational resources are considered. The comparison shows the trade-off between complexity of model and time required for training.

5. CONCLUSION

The proposed Lightweight Blockchain-Based Threat Detection (LB-TD) scheme aims to provide highly secure IoT networks leverage lightweight machine learning models working in tandem with the power of blockchain technology. The combination of Decision Trees and K-Nearest Neighbors (KNN) for detecting threats in real-time provides low latency, and low resource consumption threat logging is secure with blockchain based tamper-proof logging of classifiers. Superiority over existing models In contrast with the existing advanced model accuracy, precision, recall and F1-score are better in case of Lightweight Blockchain-Based Threat Detection (LB-TD). It also shows its scalability, managing with a higher number of devices without losing the real-time detection. The blockchain incorporation preserves the efficiency in transaction processing overhead, even with high threat levels. However, there are still challenges in the efforts of further optimizing blockchain for fast-paced networks as well as decreasing resource consumption on limited-IoT-devices. We plan to pursue in the future, better advances on machine learning approaches, improving blockchain efficiency aspects and adaptive learning for dynamic IoT networks.

Data availability statement

Data will be available on request.

Competing Interest

The authors have no competing interests to declare that are relevant to the content of this article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors

REFERENCES

- [1] Aldhaheer A, Alwahedi F, Ferrag MA, Battah A. Deep learning for cyber threat detection in IoT networks: A review. *Internet Things Cyber-Physical Syst* 2024;4:110–28. <https://doi.org/https://doi.org/10.1016/j.iotcps.2023.09.003>.
- [2] Yazdinejad A, Dehghantanha A, Parizi RM, Srivastava G, Karimipour H. Secure Intelligent Fuzzy Blockchain Framework: Effective Threat Detection in IoT Networks. *Comput Ind* 2023;144:103801. <https://doi.org/https://doi.org/10.1016/j.compind.2022.103801>.
- [3] Karie NM, Sahri NM, Haskell-Dowland P. IoT Threat Detection Advances, Challenges and Future Directions. *2020 Work. Emerg. Technol. Secur. IoT*, 2020, p. 22–9. <https://doi.org/10.1109/ETSecIoT50046.2020.00009>.
- [4] Woźniak M, Siłka J, Wiecek M, Alrashoud M. Recurrent Neural Network Model for IoT and Networking Malware Threat Detection. *IEEE Trans Ind Informatics* 2021;17:5583–94. <https://doi.org/10.1109/TII.2020.3021689>.
- [5] Kim A, Oh J, Ryu J, Lee K. A Review of Insider Threat Detection Approaches With IoT Perspective. *IEEE Access* 2020;8:78847–67. <https://doi.org/10.1109/ACCESS.2020.2990195>.
- [6] Bhargav S, Adusumilli K. Scalable Software Architecture for Dynamic Threat Detection and Mitigation in Iot 2024;18.
- [7] Ali MH, Jaber MM, Abd SK, Rehman A, Awan MJ, Damaševičius R, et al. Threat Analysis and Distributed Denial of Service (DDoS) Attack Recognition in the Internet of Things (IoT). *Electronics* 2022;11. <https://doi.org/10.3390/electronics11030494>.
- [8] Dey AK, Gupta GP, Sahu SP. A metaheuristic-based ensemble feature selection framework for cyber threat detection in IoT-enabled networks. *Decis Anal J* 2023;7:100206. <https://doi.org/https://doi.org/10.1016/j.dajour.2023.100206>.
- [9] Deng X, Zhu J, Pei X, Zhang L, Ling Z, Xue K. Flow Topology-Based Graph Convolutional Network for Intrusion Detection in Label-Limited IoT Networks. *IEEE Trans Netw Serv Manag* 2023;20:684–96. <https://doi.org/10.1109/TNSM.2022.3213807>.
- [10] Ferrag MA, Ndhlovu M, Tihanyi N, Cordeiro LC, Debbah M, Lestable T, et al. Revolutionizing Cyber Threat Detection with Large Language Models: A Privacy-Preserving BERT-Based Lightweight Model for IoT/IIoT Devices. *IEEE Access* 2024;12:23733–50. <https://doi.org/10.1109/ACCESS.2024.3363469>.
- [11] Razib MA, Javeed D, Khan MT, Alkanhel R, Muthanna MSA. Cyber Threats Detection in Smart Environments Using SDN-Enabled DNN-LSTM Hybrid Framework. *IEEE Access* 2022;10:53015–26. <https://doi.org/10.1109/ACCESS.2022.3172304>.

- [12] Villegas-Ch W, Gutierrez R, Sánchez-Salazar I, Mera-Navarrete A. Adaptive Security Framework for the Internet of Things: Improving Threat Detection and Energy Optimization in Distributed Environments. *IEEE Access* 2024;12:157924–44. <https://doi.org/10.1109/ACCESS.2024.3486983>.
- [13] Wang C, Xu D, Li Z, Niyato D. Effective Intrusion Detection in Highly Imbalanced IoT Networks With Lightweight S2CGAN-IDS. *IEEE Internet Things J* 2024;11:15140–51. <https://doi.org/10.1109/JIOT.2023.3342638>.
- [14] Koca M, Avci I. A Novel Hybrid Model Detection of Security Vulnerabilities in Industrial Control Systems and IoT Using GCN+LSTM. *IEEE Access* 2024;12:143343–51. <https://doi.org/10.1109/ACCESS.2024.3466391>.
- [15] Sharmeen S, Huda S, Abawajy JH, Ismail WN, Hassan MM. Malware Threats and Detection for Industrial Mobile-IoT Networks. *IEEE Access* 2018;6:15941–57. <https://doi.org/10.1109/ACCESS.2018.2815660>.
- [16] Li R, Li Q, Zhou J, Jiang Y. ADRIoT: An Edge-Assisted Anomaly Detection Framework Against IoT-Based Network Attacks. *IEEE Internet Things J* 2022;9:10576–87. <https://doi.org/10.1109/JIOT.2021.3122148>.
- [17] Liu Y, Zhang X, Huo L, Wu J, Guizani M. Swarm Learning and Knowledge Distillation Empowered Self-Driving Detection Against Threat Behavior for Intelligent IoT. *IEEE Trans Mob Comput* 2024;23:7117–34. <https://doi.org/10.1109/TMC.2023.3330514>.